

Week 3 Project Deliverables:

Banking Data Pipeline Documentation

1. Environment Setup & Project Structure

To build the pipeline, initialize your workspace directory with the exact structural components required for isolation:

```
Banking_Data_Pipeline/  
|  
|—— raw_data/  
|—— processed/  
|   |—— clean_transactions.csv  
|—— quarantine/  
|   |—— quarantine_transactions.csv  
|   |—— fraud_transactions.csv  
|—— reports/  
|—— logs/  
|—— sql/  
|   |—— create_schemas.sql  
|   |—— create_tables.sql  
|   |—— analysis_queries.sql  
|—— src/  
|   |—— extract.py  
|   |—— validate.py  
|   |—— transform.py  
|   |—— fraud.py  
|   |—— load.py  
|   |—— database.py  
|   |—— logger.py  
|   |—— config.py  
|   |—— main.py  
|—— screenshots/  
|—— .env  
|—— .gitignore  
|—— requirements.txt  
|—— README.md
```

Run the following terminal commands to install core dependencies safely inside environment:

```
pip install pandas psycpg2-binary python-dotenv
```

2. Database Configuration

Create an environment configuration file named `.env` in your root workspace. This hides private credentials from your public GitHub repository:

```
DB_HOST = localhost
DB_PORT = 5432
DB_NAME = banking_dw
DB_USER = postgres
DB_PASSWORD = your_secure_password
```

3. Database Schema

Execute the following foundational DDL blocks to structure data warehouse:

Schema Isolation Layer (`sql/create_schemas.sql`)

```
CREATE SCHEMA IF NOT EXISTS staging;

CREATE SCHEMA IF NOT EXISTS warehouse;

CREATE SCHEMA IF NOT EXISTS audit;
```

Relational Architecture Layout (`sql/create_tables.sql`)

```
-- create table transactions_staging inside staging schema
CREATE TABLE staging.transactions_staging(
    transaction_id VARCHAR(50),
    account_number VARCHAR(50),
    customer_id VARCHAR(50),
    transaction_datetime TIMESTAMP,
    transaction_type VARCHAR(50),
    amount NUMERIC(18,2),
    branch_code VARCHAR(20),
    region VARCHAR(20),
```

```
channel VARCHAR(20),  
status VARCHAR(20),  
fraud_flag VARCHAR(5)  
);
```

-- CREATE DIMENSION TABLES

-- i) Customer Dimension

```
CREATE TABLE IF NOT EXISTS warehouse.dim_customer (  
    customer_key SERIAL PRIMARY KEY,  
    customer_id VARCHAR(50) UNIQUE,  
    account_number VARCHAR(50)  
);
```

-- ii) Branch Dimension

```
CREATE TABLE IF NOT EXISTS warehouse.dim_branch(  
    branch_key SERIAL PRIMARY KEY,  
    branch_code VARCHAR(20) UNIQUE,  
    region VARCHAR(50)  
);
```

-- iii) Date Dimension

```
CREATE TABLE IF NOT EXISTS warehouse.dim_date(  
    date_key INT PRIMARY KEY,  
    full_date DATE UNIQUE ,  
    year INT,  
    quarter INT,  
    month INT,  
    month_name VARCHAR(20),  
    day INT,  
    day_of_week VARCHAR(20)  
);
```

-- Create Fact Table

```
CREATE TABLE IF NOT EXISTS warehouse.fact_transactions(  
  
    transaction_key SERIAL PRIMARY KEY,  
  
    transaction_id VARCHAR(50) UNIQUE,  
  
    customer_key INT REFERENCES warehouse.dim_customer(customer_key) ,  
  
    branch_key INT REFERENCES warehouse.dim_branch(branch_key),  
  
    date_key INT REFERENCES warehouse.dim_date(date_key),  
  
    transaction_type VARCHAR(50),  
  
    channel VARCHAR(50),  
  
    amount NUMERIC(18,2),  
  
    status VARCHAR(50),  
  
    fraud_flag VARCHAR(5)  
  
);
```

-- Create Audit Table

```
CREATE TABLE IF NOT EXISTS audit.etl_run_log(  
  
    run_id SERIAL PRIMARY KEY,  
  
    pipeline_name VARCHAR(100),  
  
    start_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    end_time TIMESTAMP,  
  
    records_extracted INT DEFAULT 0,
```

```
records_valid INT DEFAULT 0,

records_rejected INT DEFAULT 0,

fraud_records INT DEFAULT 0,

status VARCHAR(50),

error_message TEXT

);
```

4. ETL Execution Framework

The execution layer is divided across independent modules to strictly adhere to the single-responsibility design pattern:

- **Extract Engine (src/extract.py):**
 - It scans the raw_data/ folder to safely capture transactions from North, South, and West regions.
 - It combines these different CSV files into one clean dataset in memory for the next steps.
- **Validation Guard (src/validate.py):**
 - It drops missing data, blank IDs, or zero amounts to keep the pipeline from crashing.
 - It routes bad rows directly into the quarantine files and staging database tables.
- **Transformer (src/transform.py):** Standardizes dates, handles character case corrections, and patches missing data defaults
- **Fraud Firewall (src/fraud.py):** It scans transactions for security risks like massive payments above \$10,000.
- **Bulk Database Loader (src/load.py):**
 - It moves data into PostgreSQL quickly using fast database copy tools instead of slow single entries.
 - It matches customer and branch codes to create the final data tables without duplication
- **src/database.py:** Manages reusable PostgreSQL connections and executes fast bulk queries safely.
- **src/logger.py:** Initialises automated runtime logging to keep track of pipeline errors and progress.
- **src/config.py:** Loads system credentials and dynamic directory folder path settings securely.
- **src/main.py :** Orchestrates the entire end-to-end ETL execution flow while writing logs to the audit table

Trigger the completed orchestration matrix sequence directly from your terminal using:

```
python -m src.main OR
```

```
python src/main.py
```

5. Data Quality Checks & Guardrails

Your custom validation engine applies strict corporate safety logic to filter out corrupt parameters. The rules operate as follows:

Rule ID	What It Check	Target Column	Simple Logic Rule	What Happens If It Fails
DQ-01	Missing Data	transaction_id	Cannot be empty or blank.	Blocked → Saved into quarantine_transactions.csv
DQ-02	Customer Check	customer_id	Cannot be empty or blank.	Blocked → Saved into quarantine_transactions.csv
DQ-03	Wrong Amount	amount	Must be a number greater than 0.	Blocked → Saved into quarantine_transactions.csv
DQ-04	Bad Date Format/Future Date	transaction_datetime	Must be a real, valid date and time.	Blocked → Saved into quarantine_transactions.csv
DQ-05	High Risk Flag	amount	Checks if the amount is over \$10,000.	Flagged → Changes fraud_flag to 'YES' and saves to fraud_transactions.csv

Data Verification Checks After Loading

After loading the data into PostgreSQL, we run three simple checks to make sure everything worked perfectly:

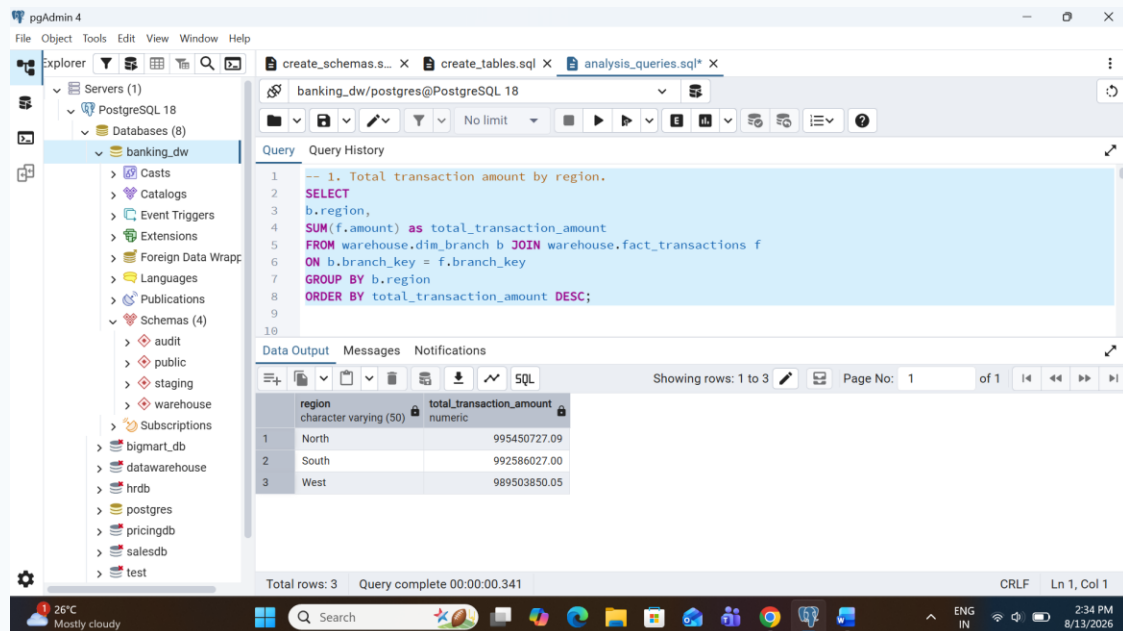
- **The Balancing Check:** We verify that the total rows we read from the raw CSV files match exactly with our clean data plus our rejected data.

- **The Duplicate Check:** We scan the final tables to make sure no transaction ID was accidentally saved twice.
- **The Missing Links Check:** We double-check that no transaction is missing its connecting links to the customer, branch, or date tables.

6. SQL Analysis & Audit Verification

Run these analytic verification queries directly inside pgAdmin to generate submission report stats:

```
-- 1. Total transaction amount by region.
SELECT
b.region,
SUM(f.amount) as total_transaction_amount
FROM warehouse.dim_branch b JOIN warehouse.fact_transactions f
ON b.branch_key = f.branch_key
GROUP BY b.region
ORDER BY total_transaction_amount DESC;
```



The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show 'PostgreSQL 18' > 'Databases (8)' > 'banking_dw'. The 'Query' editor is active, displaying the SQL query for 'Total transaction amount by region'. The 'Data Output' tab shows the results of the query, which are displayed in a table with 3 rows and 2 columns: 'region' and 'total_transaction_amount'.

region	total_transaction_amount
North	995450727.09
South	992586027.00
West	989503850.05

The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.341'.

```
-- 2. Total number of transactions by channel.
SELECT channel,
COUNT(*) as total_transactions
FROM warehouse.fact_transactions
GROUP BY channel
ORDER BY total_transactions DESC;
```

pgAdmin 4

File Object Tools Edit View Window Help

Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (8)
 - banking_dw
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas (4)
 - audit
 - public
 - staging
 - warehouse
 - Subscriptions
 - bigmart_db
 - datawarehouse
 - hrdb
 - postgres
 - pricingdb
 - salesdb
 - test

Query

```
-- 2. Total number of transactions by channel.
SELECT channel,
COUNT(*) AS total_transactions
FROM warehouse.fact_transactions
GROUP BY channel
ORDER BY total_transactions DESC;
```

Data Output

	channel character varying (50)	total_transactions bigint
1	BRANCH	49898
2	ATM	49891
3	IMPS	49856
4	NEFT	49537
5	RTGS	49429
6	UPI	49326

Total rows: 6 Query complete 00:00:00.147

26°C Mostly cloudy

Search

ENG IN

2:35 PM 8/13/2026

-- 3. Successful vs failed transactions.

```
SELECT status,
COUNT(*) AS total_transactions
FROM warehouse.fact_transactions
GROUP BY status
ORDER BY total_transactions DESC
```

pgAdmin 4

File Object Tools Edit View Window Help

Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (8)
 - banking_dw
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas (4)
 - audit
 - public
 - staging
 - warehouse
 - Subscriptions
 - bigmart_db
 - datawarehouse
 - hrdb
 - postgres
 - pricingdb
 - salesdb
 - test

Query

```
-- 3. Successful vs failed transactions.
SELECT status,
COUNT(*) AS total_transactions
FROM warehouse.fact_transactions
GROUP BY status
ORDER BY total_transactions DESC;
```

Data Output

	status character varying (50)	total_transactions bigint
1	Success	149041
2	Failed	148896

Total rows: 2 Query complete 00:00:00.155

26°C Mostly cloudy

Search

ENG IN

2:36 PM 8/13/2026

-- 4. Top 10 branches by transaction value.


```

SELECT
b.branch_code,
SUM(f.amount) AS total_value
FROM warehouse.dim_branch b JOIN warehouse.fact_transactions f
ON b.branch_key = f.branch_key
GROUP BY b.branch_code
ORDER BY total_value DESC
LIMIT 10;

```

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including servers, databases, and schemas. The central pane shows a SQL query being executed. The right pane displays the query results in a table format.

Query:

```

SELECT
b.branch_code,
SUM(f.amount) AS total_value
FROM warehouse.dim_branch b JOIN warehouse.fact_transactions f
ON b.branch_key = f.branch_key
GROUP BY b.branch_code
ORDER BY total_value DESC
LIMIT 10;

```

Data Output:

branch_code	total_value
WBR019	41382285.36
NBR012	41033864.66
NBR017	40961498.25
NBR015	40807086.59
SBR024	40698984.90
SBR020	40692020.12
SBR006	40635309.36
NBR008	40603212.89
SBR001	40514434.69
SBR003	40383305.32

Total rows: 10 Query complete 00:00:00.201

```

-- 5. Daily transaction volume.
SELECT dt.full_date,
COUNT(f.transaction_id) AS transaction_total_volume
FROM warehouse.dim_date as dt JOIN warehouse.fact_transactions as f
ON dt.date_key = f.date_key
GROUP BY dt.full_date
ORDER BY transaction_total_volume DESC;

```

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (8)
 - banking_dw
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas (4)
 - audit
 - public
 - staging
 - warehouse
 - Subscriptions
 - bigmart_db
 - datawarehouse
 - hrdb
 - postgres
 - pricingdb
 - salesdb
 - test
 - Login/Group Roles
 - Tablespaces

Query

```
-- 5. Daily transaction volume.
SELECT dt.full_date,
COUNT(f.transaction_id) AS transaction_total_volume
FROM warehouse.dim_date dt JOIN warehouse.fact_transactions as f
ON dt.date_key = f.date_key
GROUP BY dt.full_date
ORDER BY transaction_total_volume DESC;
```

Data Output

	full_date date	transaction_total_volume bigint
1	2025-02-13	905
2	2025-07-04	896
3	2025-08-24	880
4	2025-08-12	877
5	2025-02-07	875
6	2025-08-16	875
7	2025-07-27	875
8	2025-10-23	874
9	2025-10-21	871
10	2025-08-06	868

Total rows: 365 Query complete 00:00:00.176

Showing rows: 1 to 365 Page No: 1 of 1

26°C Mostly cloudy 2:39 PM 8/13/2026

```
-- 6. Monthly transaction amount.
SELECT dt.year,
dt.month_name,
SUM(f.amount) AS monthly_transaction_amount
FROM warehouse.dim_date dt JOIN warehouse.fact_transactions f
ON dt.date_key = f.date_key
GROUP BY dt.year, dt.month_name
ORDER BY monthly_transaction_amount DESC;
```

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (8)
 - banking_dw
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas (4)
 - audit
 - public
 - staging
 - warehouse
 - Subscriptions
 - bigmart_db
 - datawarehouse
 - hrdb
 - postgres
 - pricingdb
 - salesdb
 - test
 - Login/Group Roles
 - Tablespaces

Query

```
SELECT dt.year,
dt.month_name,
SUM(f.amount) AS monthly_transaction_amount
FROM warehouse.dim_date dt JOIN warehouse.fact_transactions f
ON dt.date_key = f.date_key
GROUP BY dt.year, dt.month_name
ORDER BY monthly_transaction_amount DESC;
```

Data Output

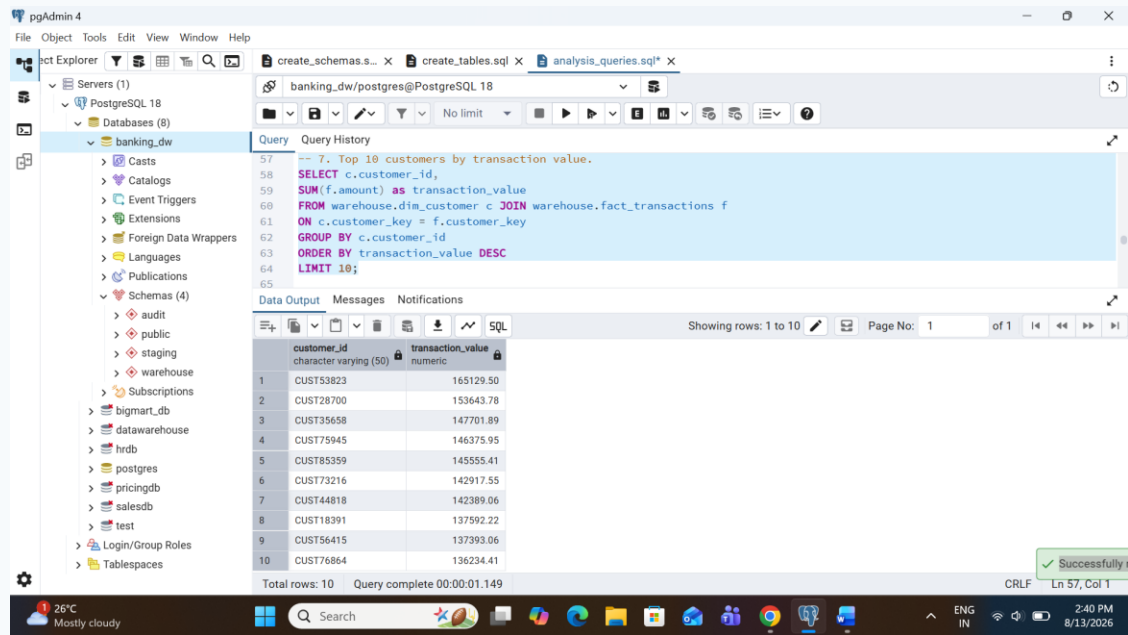
	year integer	month_name character varying (20)	monthly_transaction_amount numeric
1	2025	July	255699254.11
2	2025	January	254649274.54
3	2025	May	253009472.19
4	2025	October	251854306.61
5	2025	December	251158784.86
6	2025	March	250634420.86
7	2025	August	250358667.05
8	2025	April	246056242.76
9	2025	November	244739476.85
10	2025	June	244617194.43
11	2025	September	244596044.41
12	2025	February	230167465.47

Total rows: 12 Query complete 00:00:00.184

Showing rows: 1 to 12 Page No: 1 of 1

26°C Mostly cloudy 2:40 PM 8/13/2026

```
-- 7. Top 10 customers by transaction value.
SELECT c.customer_id,
SUM(f.amount) as transaction_value
FROM warehouse.dim_customer c JOIN warehouse.fact_transactions f
ON c.customer_key = f.customer_key
GROUP BY c.customer_id
ORDER BY transaction_value DESC
LIMIT 10;
```



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'banking_dw' database. The central pane shows a SQL query being executed. The right pane displays the results of the query in a table format.

Query:

```
-- 7. Top 10 customers by transaction value.
SELECT c.customer_id,
SUM(f.amount) as transaction_value
FROM warehouse.dim_customer c JOIN warehouse.fact_transactions f
ON c.customer_key = f.customer_key
GROUP BY c.customer_id
ORDER BY transaction_value DESC
LIMIT 10;
```

Data Output:

customer_id	transaction_value
CUST53823	165129.50
CUST28700	153643.78
CUST35658	147701.89
CUST75945	146375.95
CUST85359	145555.41
CUST73216	142917.55
CUST44818	142389.06
CUST18391	137592.22
CUST56415	137393.06
CUST76864	136234.41

Total rows: 10 Query complete 00:00:01.149

```
-- 8. Number of fraud-flagged transactions by region.
SELECT
b.region,
COUNT(*) as total_fraud_flagged
FROM warehouse.dim_branch b JOIN warehouse.fact_transactions f
ON b.branch_key = f.branch_key
WHERE f.fraud_flag = 'YES'
GROUP BY b.region
ORDER BY total_fraud_flagged DESC;
```

pgAdmin 4

Object Explorer: PostgreSQL 18, Databases (8), banking_dw

Query: Query History

```
-- 8. Number of fraud-flagged transactions by region.
SELECT
  b.region,
  COUNT(*) AS total_fraud_flagged
FROM warehouse.dim_branch b JOIN warehouse.fact_transactions f
ON b.branch_key = f.branch_key
WHERE f.fraud_flag = 'YES'
GROUP BY b.region
ORDER BY total_fraud_flagged DESC;
```

Data Output: Showing rows: 1 to 3 of 1

region	total_fraud_flagged
North	49791
South	49518
West	49349

Successfully run. Total query runtime: 175 msec. 3 rows affected.

Total rows: 3 Query complete 00:00:00.175

```
-- 9. Average transaction amount by transaction type.
SELECT transaction_type,
  ROUND(AVG(amount), 2) AS average_transaction_amount
FROM warehouse.fact_transactions
GROUP BY transaction_type;
```

pgAdmin 4

Object Explorer: PostgreSQL 18, Databases (8), banking_dw

Query: Query History

```
-- 9. Average transaction amount by transaction type.
SELECT transaction_type,
  ROUND(AVG(amount), 2) AS average_transaction_amount
FROM warehouse.fact_transactions
GROUP BY transaction_type;
```

Data Output: Showing rows: 1 to 3 of 1

transaction_type	average_transaction_amount
Deposit	9995.10
Transfer	9985.94
Withdrawal	10000.50

Successfully run. Total query runtime: 147 msec. 3 rows affected.

Total rows: 3 Query complete 00:00:00.147

```
-- 10. Highest-value transactions.
SELECT
  f.transaction_id,
```

```

f.amount,
b.region
FROM warehouse.dim_branch b JOIN warehouse.fact_transactions f
ON b.branch_key = f.branch_key
ORDER BY f.amount DESC
LIMIT 10;

```

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'banking_dw' database. The main pane shows a SQL query being executed. The query is as follows:

```

-- 10. Highest-value transactions.
SELECT
f.transaction_id,
f.amount,
b.region
FROM warehouse.dim_branch b JOIN warehouse.fact_transactions f
ON b.branch_key = f.branch_key
ORDER BY f.amount DESC
LIMIT 10;

```

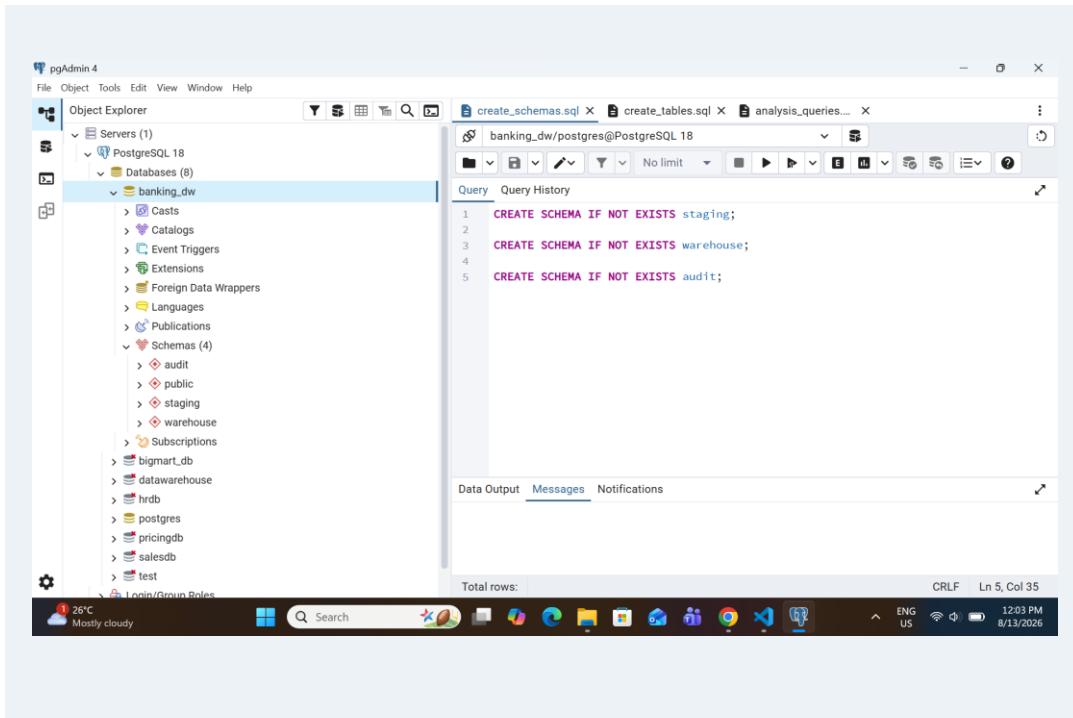
The results are displayed in a table with the following columns: transaction_id, amount, and region. The table shows the top 10 transactions by amount.

transaction_id	amount	region
WTX1006129	19999.89	West
WTX1053712	19999.84	West
STX1048688	19999.80	South
NTX1053559	19999.64	North
NTX1031137	19999.64	North
STX1019409	19999.61	South
NTX1000123	19999.53	North
STX1050616	19999.46	South
NTX1009076	19999.38	North
STX1096811	19999.20	South

A status message at the bottom right indicates: "Successfully run. Total query runtime: 222 msec. 10 rows affected."

7. Screenshots Checklist

Database Structure Screenshot (Showing staging, warehouse, and audit custom schemas layout)



Tables & Row Counts Screenshot (Grid run output proving record placement counts)

The screenshot displays the pgAdmin 4 interface. On the left, the Object Explorer shows a tree structure with 'Servers (1)' containing 'PostgreSQL 18', which has 'Databases (8)' including 'banking_dw'. The 'Query' tab is active, showing a SQL query that counts records from several tables in the 'warehouse' schema. The 'Data Output' tab shows the results of the query in a table grid.

Query:

```
14 SELECT 'staging.transactions_staging' AS tbl, COUNT(*) FROM staging.transactions_staging
15 UNION ALL
16 SELECT 'warehouse.fact_transactions', COUNT(*) FROM warehouse.fact_transactions
17 UNION ALL
18 SELECT 'warehouse.dim_customer', COUNT(*) FROM warehouse.dim_customer
19 UNION ALL
20 SELECT 'warehouse.dim_branch', COUNT(*) FROM warehouse.dim_branch
21 UNION ALL
22 SELECT 'warehouse.dim_date', COUNT(*) FROM warehouse.dim_date;
```

Data Output:

	tbl	count
	text	bigint
1	warehouse.dim_branch	75
2	warehouse.dim_date	365
3	warehouse.dim_customer	86679
4	warehouse.fact_transactions	297937
5	staging.transactions_staging	297937

Total rows: 5 Query complete 00:00:00.237 CRLF Ln 14, Col 1

26°C Mostly cloudy Search ENG US 12:05 PM 8/13/2026

Audit Log Table Verification Screenshot (Showing a row containing SUCCESS state row counts)

pgAdmin 4

File Object Tools Edit View Window Help

create_schemas.s... create_tables.sql* analysis_queries...

banking_dw/postgres@PostgreSQL 18

No limit

Servers (1)

- PostgreSQL 18
 - Databases (8)
 - banking_dw
 - bigmart_db
 - datawarehouse
 - hrdb
 - postgres
 - pricingdb
 - salesdb
 - test
 - Login/Group Roles
 - Tablespaces

Query Query History

error_message TEXT

);

SELECT * FROM audit.etl_run_log

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

run_id [PK] integer	pipeline_name character varying (100)	start_time timestamp without time zone	end_time timestamp without time zone	records_extracted integer	records_valid integer	records_rejected integer	fraud_records integer
1	banking_dw_load_pipeli...	2026-08-13 12:07:03.574519	2026-08-13 12:07:58.89038	297937	297937	0	

Total rows: 1 Query complete 00:00:00.163

26°C Mostly cloudy

Search

ENG US 12:08 PM 8/13/2026

pgAdmin 4

File Object Tools Edit View Window Help

create_schemas.s... create_tables.sql* analysis_queries...

banking_dw/postgres@PostgreSQL 18

No limit

Servers (1)

- PostgreSQL 18
 - Databases (8)
 - banking_dw
 - bigmart_db
 - datawarehouse
 - hrdb
 - postgres
 - pricingdb
 - salesdb
 - test
 - Login/Group Roles
 - Tablespaces

Query Query History

error_message TEXT

);

SELECT * FROM audit.etl_run_log

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

start_time timestamp without time zone	end_time timestamp without time zone	records_extracted integer	records_valid integer	records_rejected integer	fraud_records integer	status character varying (50)	error_message text
07:03.574519	2026-08-13 12:07:58.89038	297937	297937	0	148658	SUCCESS	[null]

Total rows: 1 Query complete 00:00:00.163

26°C Mostly cloudy

Search

ENG US 12:08 PM 8/13/2026

Architecture Topology Flowchart Diagram (ETL stages blueprint showing layout routing)

